

Lab 6: DiffuserCam and Wiener Deconvolution

Due Wednesday, March 18, 11:59pm

In Labs 3–5 you studied convolution, Fourier transforms, and noise individually. This lab brings all three together in a real computational camera: *DiffuserCam*. DiffuserCam is a camera that uses a diffuser—a thin, wavy piece of translucent material—instead of a lens. Each scene point is scattered into a large, complex pattern across the sensor, producing measurements that look nothing like the original scene. The key insight is that this scattering is shift-invariant: every point source produces the same pattern, just shifted to a different location. This means the imaging process is well-described by convolution with a known point spread function (PSF), and we can computationally undo it to recover recognizable images. You can read more about DiffuserCam in the original paper¹ and the DIY tutorial at waller-lab.github.io/DiffuserCam/tutorial.html.

The forward model for DiffuserCam is

$$\mathbf{y} = \text{crop}(\mathbf{h} * \mathbf{x})$$

where $\mathbf{h}$ is the PSF, $\mathbf{x}$ is the scene, $*$ denotes convolution, and the crop operator models the fact that the sensor captures only a portion of the full convolution output. You will build your own DiffuserCam, measure its PSF, implement this forward model in the frequency domain, and reconstruct images using a Wiener deconvolution filter. In the next lab, you will implement iterative algorithms to improve on the Wiener filter results.

Allowed libraries. You may use `numpy`, `scipy`, `matplotlib`, `imageio`, and `rawpy`.

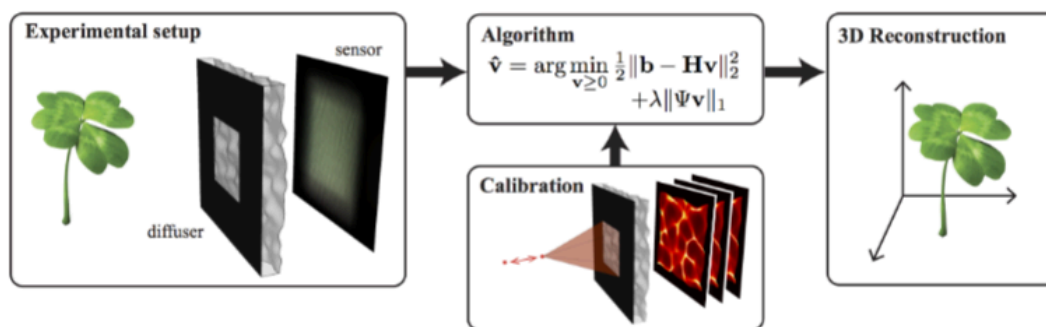


Figure 1: DiffuserCam overview: a diffuser replaces the lens, producing a coded measurement that is computationally reconstructed.

1 Building and Calibrating Your DiffuserCam

In this section you will assemble a physical DiffuserCam and capture the calibration data needed for computational reconstruction. You will need:

- A Canon R10 camera body (be careful when removing the lens!)
- A DiffuserCam adapter
- A piece of double-sided Scotch tape (this is your diffuser!)
- Black tape for making an aperture
- A mini tripod

¹Antipa et al., “DiffuserCam: Lensless Single-exposure 3D Imaging,” *Optica* 5(1), 2018.

Place the double-sided tape over the opening of the 3D-printed adapter. Use black tape to create a small aperture ($\sim 4 \times 4$ mm) on the tape. The aperture limits the angle of light entering the system and plays a role in setting the field of view of your DiffuserCam. Attach the adapter to the camera body. Be careful not to get fingerprints on the tape—this will cause smudges and ruin image quality.

Once your DiffuserCam is assembled, take some pictures and see what they look like! The measurements will appear scrambled—we will develop algorithms to unscramble them, but first we need to characterize the forward model.

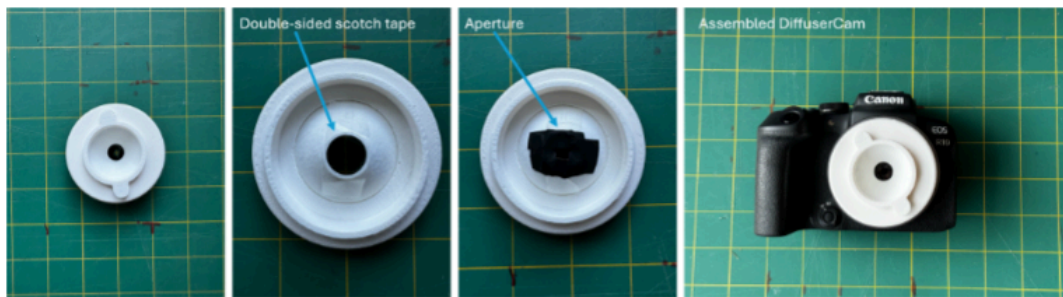


Figure 2: DiffuserCam assembly: 3D-printed adapter, double-sided tape diffuser, aperture, and mounted on camera.

Problem 1.1: Measuring the PSF.

In a dark room, set up your DiffuserCam on a tripod and place a point light source (e.g., phone flashlight) far away from the camera. Use manual mode. Set lowest ISO, auto white balance off, and adjust exposure until the PSF pattern is visible but not over-exposed. Include your PSF image.

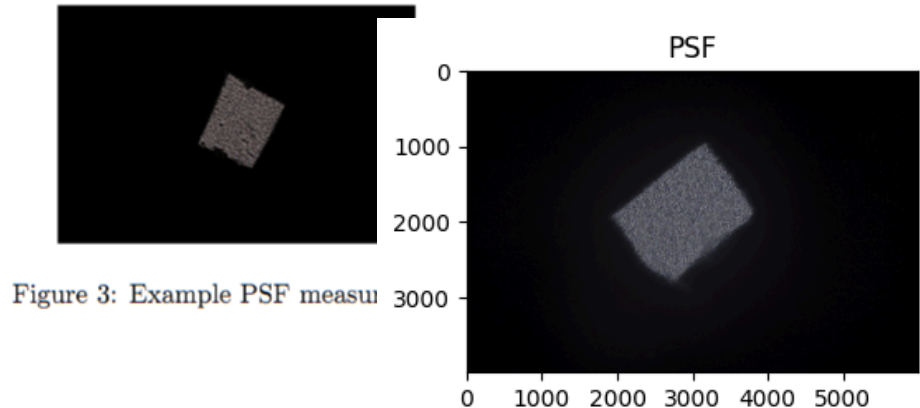
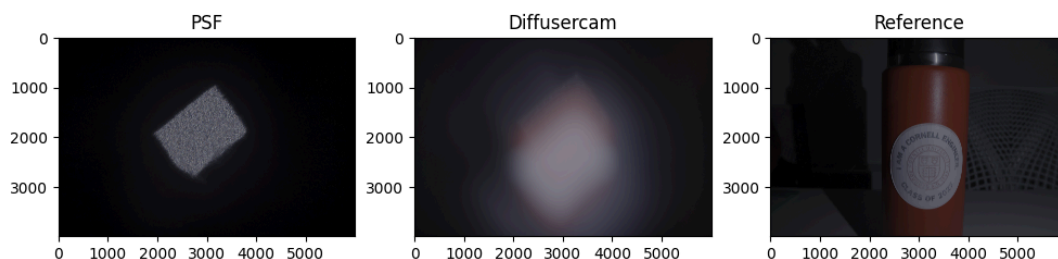


Figure 3: Example PSF measurement

Problem 1.2: Capturing a scene and visualizing your data.

Choose a scene with a dark background and a bright or self-illuminating subject (e.g., a phone screen at maximum brightness displaying a simple logo or a small object with directed lighting). Capture a DiffuserCam measurement, then replace the adapter with your regular lens and capture a lensed reference image of the same scene.

Load your PSF, DiffuserCam measurement, and lensed reference image. Plot all three side by side.



2 Implementing the Forward Model

The DiffuserCam forward model in the spatial domain is

$$\mathbf{y} = \text{crop}(\mathbf{h} * \mathbf{x})$$

To implement this efficiently, we work in the frequency domain:

$$\mathbf{y} = \text{crop}(\mathcal{F}^{-1}(\mathcal{F}(\mathbf{h}_p) \cdot \mathcal{F}(\mathbf{x}_p)))$$

where $\mathbf{h}_p = \text{pad}(\mathbf{h})$, $\mathbf{x}_p = \text{pad}(\mathbf{x})$, and $\mathcal{F}$ denotes the 2D FFT.

We must zero-pad the input image for the forward model. FFTs compute circular convolution, so padding to at least twice the original size converts this to a correct linear convolution. Padding also accounts for the fact that the diffuser scatters light from scene points outside the sensor's field of view onto the sensor—the scene is effectively larger than the measurement. The crop operator extracts the sensor-sized portion of the result. The following starter functions handle padding and cropping:

```
import numpy as np

def pad(img):
    """Zero-pad image to 2x size. (H,W,C) -> (2H,2W,C)"""
    return np.pad(img, ((img.shape[0]//2, img.shape[0]//2),
                        (img.shape[1]//2, img.shape[1]//2),
                        (0, 0)), mode='constant')

def crop(img):
    """Crop padded image back to original size. (2H,2W,C) -> (H,W,C)"""
    return img[img.shape[0]//4:-img.shape[0]//4,
              img.shape[1]//4:-img.shape[1]//4]
```

We recommend downsampling your images by 4–8× before processing to keep computation times manageable. Make sure to normalize your images before applying the forward model or any reconstruction algorithm.

Problem 2.1:

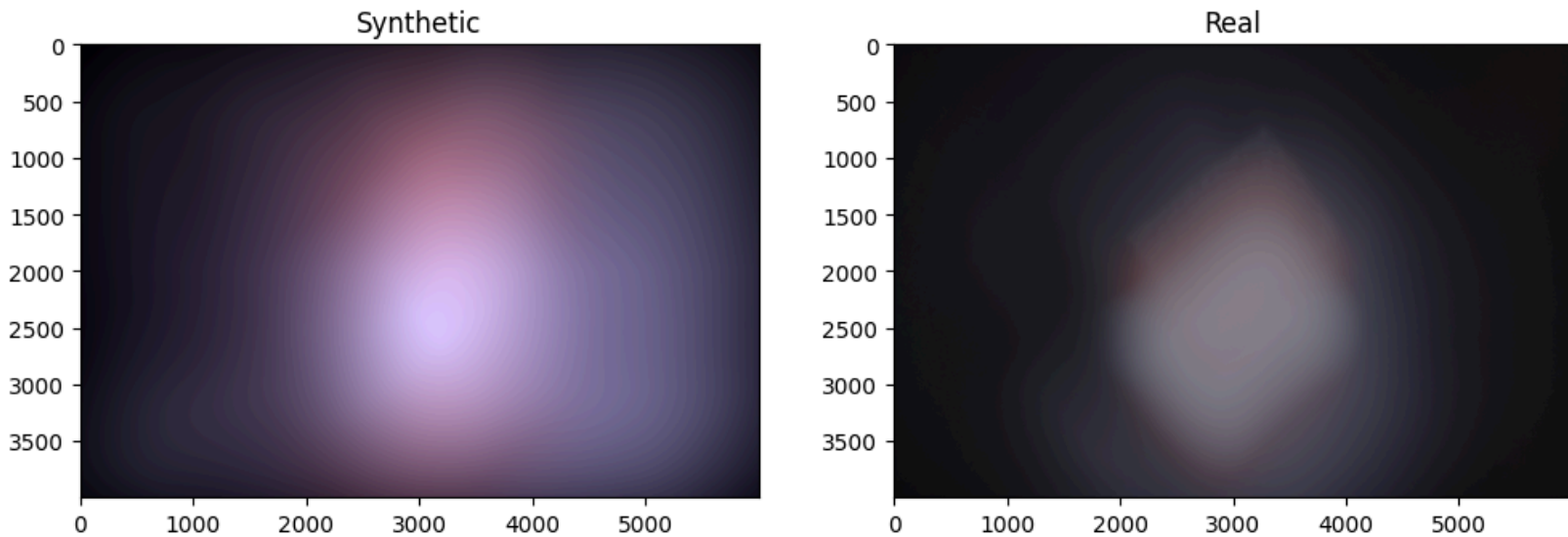
- (a) What does the crop operator correspond to physically? What would happen to your simulated measurement if you omitted it?

The crop represents the physical boundaries of the sensor. If it was omitted, the measurement would represent a sensor that captured all the light that scattered from the diffuser. The image would be twice the size and show the full convolution.

(b) Implement the DiffuserCam forward model. Your function `forward(x,h)` should:

- (a) Zero-pad both the image $\mathbf{x}$ and the PSF $\mathbf{h}$.
- (b) Compute the element-wise product of their FFTs.
- (c) Apply `ifft2`, then `ifftshift`, then crop back to the original size.
- (d) Return the absolute value of the result.

Simulate a synthetic measurement using your lensed reference image as $\mathbf{x}$ and your measured PSF as $\mathbf{h}$. Display the synthetic measurement alongside the real DiffuserCam measurement.



(c) Your synthetic and real measurements will not match perfectly. Give two reasons why they might differ.

1. The real measurement would contain noise such as photon noise and dark noise, which the synthetic model would not account for.
2. The PSF that was measured was at a single point at a specific distance. Depth variation between the PSF and the image could alter how the image looks compared to the real image.

Checkoff

Show a TA: (1) your assembled DiffuserCam, (2) your PSF image, (3) your DiffuserCam measurement, and (4) your synthetic vs. real measurement comparison.

3 Wiener Deconvolution

Now that you have a working forward model, you will implement the inverse: recovering an image from a DiffuserCam measurement. Given $Y = H \cdot X$ in the frequency domain, the simplest approach is naive deconvolution: $\hat{X}(\mathbf{f}) = Y(\mathbf{f})/H(\mathbf{f})$. This recovers X exactly when $H \neq 0$ everywhere and there is no noise, but in practice it fails catastrophically because the DiffuserCam OTF has many frequencies where $|H|$ is very small, amplifying noise without bound.

The Wiener filter addresses this by incorporating a regularization parameter τ :

$$G(\mathbf{f}) = \frac{H^*(\mathbf{f})}{|H(\mathbf{f})|^2 + \tau}$$

The reconstruction is $\hat{X}(\mathbf{f}) = G(\mathbf{f}) \cdot Y(\mathbf{f})$, and the spatial-domain estimate is $\hat{\mathbf{x}} = \mathcal{F}^{-1}\{\hat{X}\}$. Small τ approaches naive deconvolution (sharper but noisier); large τ over-smooths the result.

Problem 3.1:

- Try naive deconvolution ($\hat{X} = Y/H$) on both your synthetic and real measurements. Display the results. Why does it fail?
- Implement a function `wiener(y,h,tau)` that performs Wiener deconvolution. Your function should:
 - Zero-pad the measurement `y` and the PSF `h`.
 - Compute their FFTs.
 - Construct the Wiener filter $G = H^*/(|H|^2 + \tau)$.
 - Multiply $G \cdot Y$, apply `ifft2` and `ifftshift`, crop, and take the real part.
- Apply your Wiener deconvolution to both a synthetic measurement (using your lensed reference as `x`) and your real DiffuserCam measurement. Sweep τ in orders of 10 from 10^{-6} to 10^5 . Display all reconstructions.
- Which value of τ works best? Why might the optimal τ differ between the synthetic and real cases?

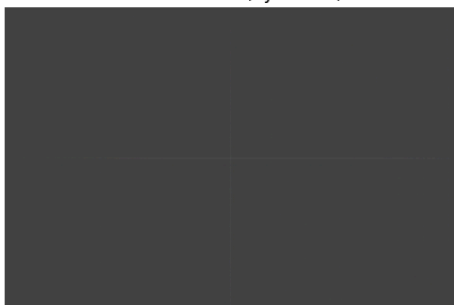
Submission

Submit the following on Gradescope:

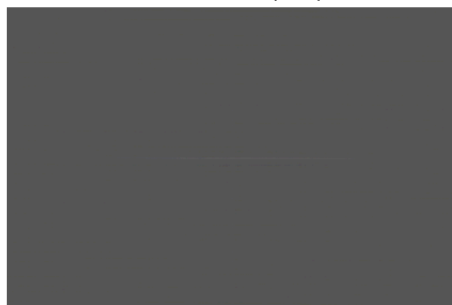
- A single PDF with answers to all problems (Sections 1–3), including all requested figures, code, and DiffuserCam images. Submit this to “Lab 6 Writeup.”
- Your code files (Python scripts, Jupyter notebook, or any format). Include a `README.md` explaining how to run your code. Submit this to “Lab 6 Code.”

a) It fails because of the divide by zero/near zero, which will cause it to destroy a lot of the image due to its numerical blowup.

Naive Deconv (Synthetic)



Naive Deconv (Real)



b) done

```
def wiener(y, h, tau):  
    y_padded = pad(y)  
    h_padded = pad(h)  
  
    Y = np.fft.fft2(y_padded, axes=(0, 1))  
    H = np.fft.fft2(h_padded, axes=(0, 1))  
  
    H_conj = np.conj(H)  
    G = H_conj / (np.abs(H)**2 + tau)  
  
    X_hat = G * Y  
    x_full = np.fft.ifftshift(np.fft.ifft2(X_hat, axes=(0,1)), axes=(0,1))  
  
    return np.real(crop(x_full))
```

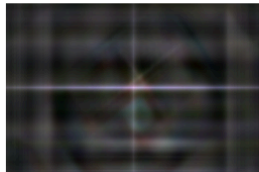
c)

Wiener Reconstructions (Real)

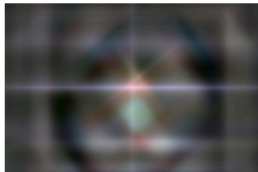
tau=1e-06



tau=1e-05



tau=1e-04



tau=1e-03



tau=1e-02



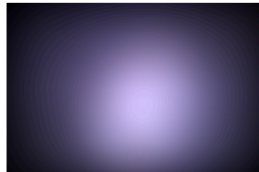
tau=1e-01



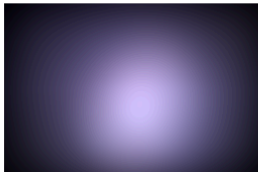
tau=1e+00



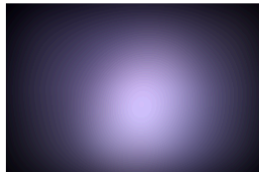
tau=1e+01



tau=1e+02



tau=1e+03



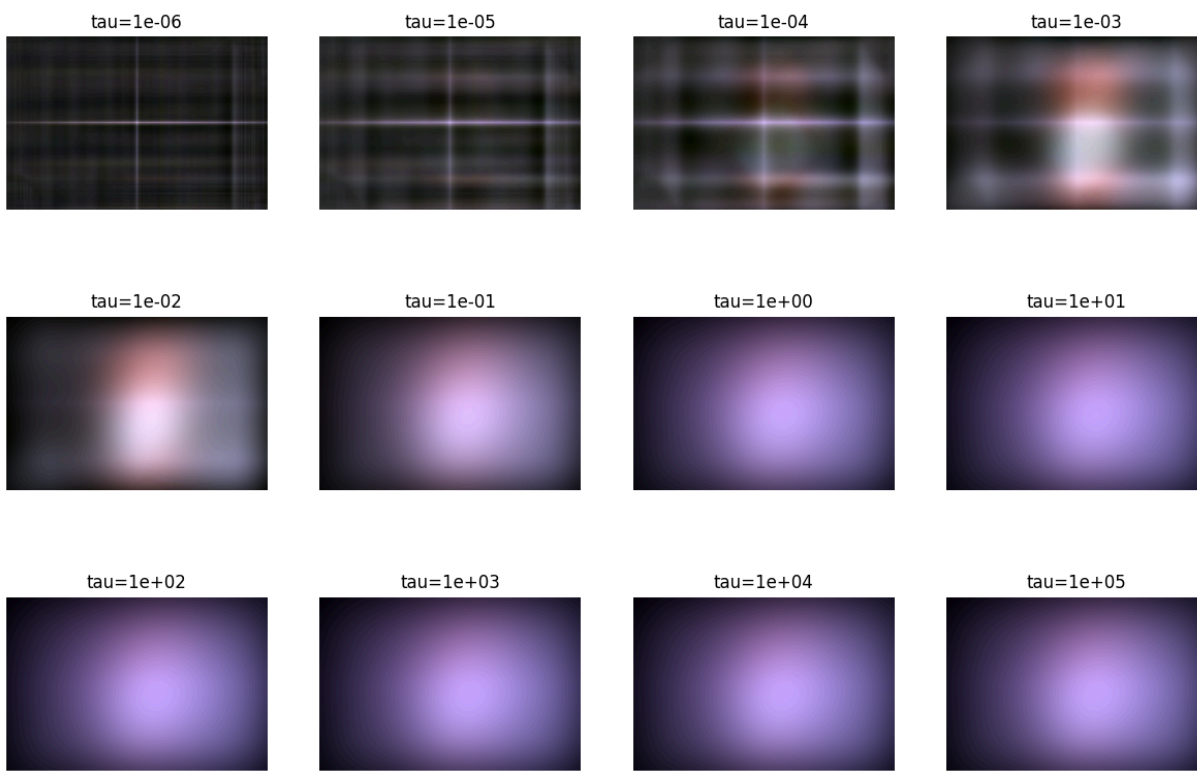
tau=1e+04



tau=1e+05



Wiener Reconstructions (Synthetic)



d) The value of τ that works best is around $1e-3$ for both, the optimal τ may be different because there is more noise present in the diffuser cam, which may need a larger τ .